

Sistema Multiagente con Orquestación, RAG, Búsqueda Web, Memoria y MCP

Emmanuel Rodriguez Rivas
José Miguel González Barrantes
Fabricio Picado Alvarado
Curso de Inteligencia Artificial
Instituto Tecnológico de Costa Rica

Resumen—El presente informe describe la arquitectura de un sistema multiagente para responder las consultas sobre apuntes del curso de Inteligencia Artificial y coordinar las fuentes de información internas y externas. La solución incluye a un orquestador central, recuperación aumentada por generación (RAG), búsqueda web, memoria conversacional, consulta transaccional mediante MCP, síntesis de información y observabilidad. El diseño implementado se separa decisión, recuperación, generación y trazabilidad para seleccionar así el flujo adecuado según la pregunta realizada.

Index Terms—sistemas multiagente, RAG, orquestación, búsqueda web, MCP, memoria conversacional, modelos de lenguaje, trazabilidad

I. INTRODUCCIÓN

El sistema propuesto resuelve el problema de responder preguntas tanto académicas como operativas mediante la arquitectura multiagente capaz de seleccionar distintas fuentes de información según se presente la necesidad a partir de lo que solicita el usuario. La solución combina agentes especializados, herramientas externas y un orquestador central que decide si una consulta debe resolverse con documentos del curso, búsqueda web, memoria conversacional, síntesis o acceso transaccional controlado [1], [2].

El componente documental se apoya en Retrieval-Augmented Generation (RAG), lo que permite generar respuestas basadas en los apuntes del curso y no únicamente en el conocimiento del modelo [3]. De forma complementaria, la arquitectura contempla búsqueda web para información externa o reciente, memoria temporal e histórica para continuidad conversacional, un agente resumidor para condensar información y un agente transaccional conectado a un servidor MCP para consultar datos ficticios bajo reglas de seguridad.

II. ARQUITECTURA DEL SISTEMA

La arquitectura se organiza en cinco niveles principales: la interfaz de usuario, capa de comunicación, orquestador, agentes especializados y herramientas de soporte. La interfaz recibe la consulta y presenta la respuesta final. La capa de comunicación expone el flujo conversacional hacia el backend. El orquestador analiza la consulta y coordina el agente correspondiente y el más óptimo para la tarea.

Los agentes se encargan de cubrir distintos tipos de consulta. El agente RAG atiende preguntas sobre los apuntes del curso; el agente de búsqueda web recupera fuentes externas; el agente

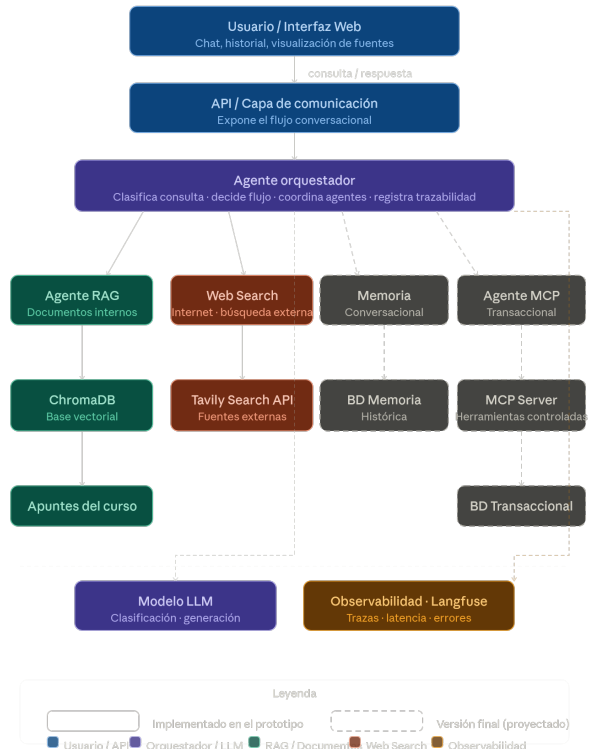


Figura 1. Arquitectura general del sistema multiagente con orquestación, RAG, búsqueda web, memoria, MCP y observabilidad.

de memoria gestiona contexto temporal e histórico; el agente resumidor sintetiza información; y el agente transaccional interactúa con herramientas MCP para consultar los datos ficticios.

La Fig. 1 muestra la vista general del sistema. El orquestador funciona como punto de coordinación entre los agentes y las herramientas externas, mientras que la observabilidad registra decisiones, fuentes utilizadas, llamadas a herramientas y errores relevantes.

III. MODELO DE ORQUESTACIÓN

El orquestador recibe una consulta y determina qué flujo debe ejecutarse. Para ello, evalúa si la pregunta requiere algún conocimiento documental, información externa, memoria

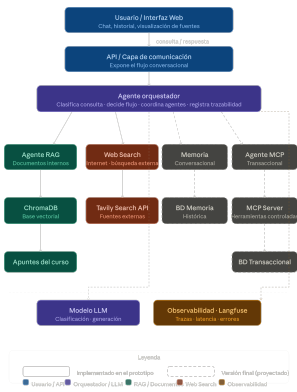


Figura 2. Flujo de decisión del orquestador para coordinar agentes especializados según la naturaleza de la consulta.

conversacional, síntesis o consulta transaccional. Esta decisión evita que la interfaz o los agentes especializados asuman responsabilidades de coordinación, separando correctamente las funciones específicas de cada agente.

Cuando la pregunta se relaciona con los conceptos, definiciones o explicaciones que se encuentran en los apuntes, el orquestador dirige el flujo hacia RAG. Cuando solicita información reciente o externa, activa búsqueda web. Si depende del historial, se considera memoria; si requiere condensar información, se activa síntesis; y si involucra datos ficticios transaccionales, se dirige hacia el flujo MCP bajo restricciones de seguridad. No obstante estos últimos elementos son implementados hasta el final.

IV. FLUJO DE PROCESAMIENTO DE CONSULTAS

IV-A. Consultas Documentales Mediante RAG

Preprocesamiento de documentos Para construir la base documental del sistema RAG, se utilizaron los apuntes del curso en formato PDF como fuente principal de información. Estos documentos fueron procesados antes de ser almacenados en la base vectorial, con el objetivo de convertir su contenido en fragmentos consultables (chunks) por el agente RAG. Este enfoque sigue la idea general de la generación aumentada por recuperación, donde un modelo generativo utiliza información recuperada desde una memoria externa o índice documental para producir respuestas más fundamentadas [3].

El proceso inicia con la extracción del texto de cada PDF ubicado en el directorio de apuntes del proyecto. Luego, el contenido extraído pasa por una etapa de limpieza básica, donde se eliminan espacios innecesarios, saltos de línea redundantes y caracteres que pueden afectar la calidad del texto recuperado. Esta etapa es importante porque los documentos PDF pueden contener texto con cortes, errores de codificación o irregularidades generadas durante la extracción.

Después, el texto se divide en fragmentos o *chunks*. Para esta primera versión se utilizó una estrategia de fragmentación fija, definiendo un tamaño de fragmento y un solapamiento entre fragmentos consecutivos. El solapamiento permite conservar parte del contexto entre segmentos, reduciendo el

riesgo de perder información relevante cuando una idea queda separada entre dos fragmentos.

Cada fragmento se almacena junto con metadatos pertenecientes al documento original. Entre estos metadatos se incluyen el nombre del archivo, la página, la semana del curso, la sección detectada y otros datos útiles solicitados para rastrear la fuente de la información. Esto permite que, cuando el sistema recupera un fragmento, también pueda mostrar al usuario de dónde proviene la información utilizada para generar la respuesta.

Usuario → Orquestador → RAGAgent →
 Embeddings → ChromaDB → Chunks
 relevantes → LLM → Respuesta con fuentes

En este flujo, la pregunta del usuario no se envía directamente al modelo de lenguaje. Primero, el agente RAG utiliza la consulta para buscar fragmentos relevantes dentro de la base vectorial construida a partir de los apuntes del curso, como se explicó anteriormente. Para ello, la pregunta se transforma en una representación semántica mediante el mismo modelo de *embeddings* utilizado durante la indexación de documentos. El siguiente paso es que esta representación se compara contra los vectores almacenados en ChromaDB para recuperar los fragmentos con mayor similitud semántica, es decir cual de los chunks tiene un mayor nivel de "match".

Una vez recuperados los fragmentos relevantes, el sistema construye un *prompt* que incluye la pregunta original y el contexto obtenido desde los apuntes. Este contexto es entregado al modelo generativo, el cual produce una respuesta basada únicamente en los fragmentos recuperados, esto debido a que desde un principio el modelo ya se le definieron reglas donde se le menciona explícitamente no usar ni inventar información que no este registrada en los apuntes de clase. Finalmente, la respuesta se complementa con las fuentes utilizadas, incluyendo archivo, página, semana y sección, de modo que el usuario pueda verificar la procedencia de la información gracias al registro y al uso de los metadatos.

En pocas palabras, flujo permite que el sistema primero busque información relevante en los apuntes y luego use esa información para generar la respuesta. De esta forma, el modelo no responde únicamente con su conocimiento interno, sino que se apoya en fragmentos recuperados desde los documentos del curso [3]. Además, el uso de *embeddings* permite encontrar fragmentos relacionados con la pregunta aunque no tengan exactamente las mismas palabras escritas por el usuario, sino que sea seleccionado por la similitud semántica. [4].

IV-B. Consultas Externas Mediante Web Search

Las consultas externas se atienden cuando la pregunta requiere información reciente, externa o solicitada explícitamente desde la web:

Usuario → Orquestador → Web Search →
 Tavily → LLM → Respuesta

Para el uso de búsqueda web se requiere una justificación generada por el proceso de orquestación, mediante esta

condición se evita activar fuentes externas como mecanismo default y permite mantener control sobre cuándo el sistema va a abandonar la base documental.

Los resultados recuperados se normalizan y se entregan como contexto al modelo de lenguaje. La respuesta debe basarse en las fuentes obtenidas, reduciendo el riesgo de introducir información la cual no es respaldada.

IV-C. Observabilidad mediante Langfuse Cloud

La observabilidad del sistema se implementó con Langfuse Cloud para poder revisar qué ocurre internamente cuando el usuario realiza una consulta. En el caso del flujo principal de orquestación y RAG, se registra la pregunta del usuario, la decisión tomada por el orquestador, el agente seleccionado, la herramienta utilizada, los fragmentos recuperados desde ChromaDB, la llamada al modelo de lenguaje, la respuesta generada y la latencia del proceso.

Para las consultas documentales, el *RAGAgent* utiliza *RAG-Tool* y ChromaDB para buscar información relacionada con la pregunta dentro de los apuntes del curso. La traza guarda los fragmentos recuperados junto con datos como el archivo, la página, la semana, la sección y la distancia de similitud. Después de eso, también se registra la llamada al modelo Gemini, donde se envía la pregunta junto con el contexto recuperado para generar la respuesta final. Aunque se mencionó anteriormente, algunos de los datos crudos que se pueden observar mediante Langfuse son: Entrada del usuario, Agente que recibió la solicitud, Herramientas utilizadas, Fragmentos recuperados por el RAG, Llamadas al modelo de lenguaje, Latencia, Errores.

V. PRESENTE Y FUTURAS DECISIONES ARQUITECTÓNICAS E INTEGRACIÓN TECNOLÓGICA

El diseño usa un orquestador central para concentrar la decisión del flujo y así mantener los agentes especializados con sus responsabilidades delimitadas. Esta estructura permite integrar agentes documentales, externos, conversacionales, de síntesis y transaccionales bajo el mismo esquema de coordinación.

La arquitectura separa la recuperación y generación. En RAG, primero se recuperan fragmentos documentales y luego se genera la respuesta. En Web Search, primero se recuperan fuentes externas y después se redacta la respuesta con base en ellas. El mismo principio aplica para memoria y MCP, donde el modelo trabaja sobre contexto previamente recuperado o datos controlados.

Gemini se integra para apoyar clasificación y generación de respuestas [5]. Tavily se utiliza para búsqueda externa orientada a aplicaciones con agentes [6]. ChromaDB funciona como infraestructura vectorial para consultar representaciones semánticas de documentos [7]. FastAPI actúa como capa de comunicación entre interfaz y backend [8]. Langfuse permite registrar trazas, decisiones, llamadas a herramientas y errores dentro del flujo [9].

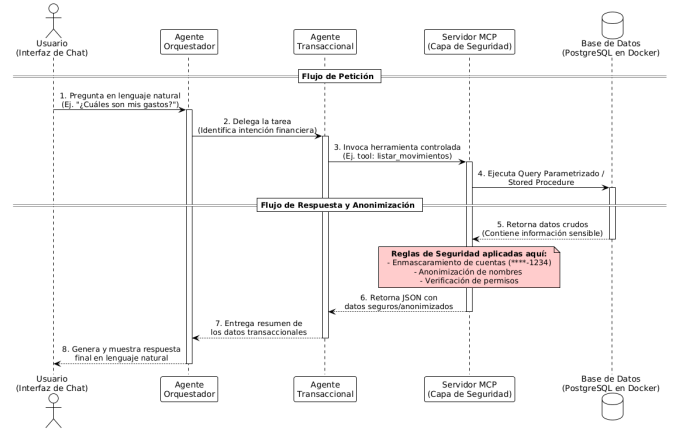


Figura 3. Diagrama de arquitectura segura utilizando un MCP Server.

V-A. Incorporación de componentes para la segunda entrega

Actualmente, el grupo también se encuentra trabajando en el agente resumidor, el servidor MCP y la base de datos que será utilizada para almacenar información persistente del sistema. El agente resumidor se utilizará para condensar información extensa que venga de documentos, resultados web o historial conversacional. Su función será generar respuestas más cortas y organizadas cuando el sistema obtenga mucho contexto o cuando el usuario solicite un resumen directamente.

Tomando en cuenta la Fig.3, el agente transaccional se incorporará mediante un servidor MCP, el cual permitirá consultar datos ficticios o controlados de forma segura. Este componente se manejará bajo reglas específicas, de manera que el sistema pueda acceder a herramientas externas sin exponer operaciones no autorizadas.

La base de datos transaccional será realizada en PostgreSQL, la cual se implementará utilizando la imagen de dicho motor de base de datos en Docker en un contenedor aislado. Esta base de datos será llenada utilizando un script de Python; generando el volumen de datos ficticios necesarios para las pruebas requeridas para el sistema.

El servidor MCP se desarrollará en Python utilizando el SDK desarrollado por Anthropic, el cual se encargará de ser un servicio intermediario, exponiendo herramientas de lectura de datos mediante la ejecución exclusiva de procedimientos almacenados en la base de datos. Además, mencionar que bajo ninguna circunstancia el agente tendrá permisos para ejecutar código SQL dinámico o libre.

Seguidamente, el MCP obtiene los datos crudos desde la base de datos, luego aplicará políticas de seguridad en tiempo de ejecución para anonimizar los datos sensibles y enmascarar información crítica. Solamente después de aplicar este filtro a los datos, el MCP estructurará los datos en formato JSON y los va a retornar al agente transaccional que realizó la petición, garantizando así el principio de mínimo privilegio y seguridad por diseño.

Finalmente, la memoria histórica persistente se apoyará en una base de datos para conservar información relevante

de interacciones anteriores. Esto ayudará a que el sistema pueda mantener continuidad entre consultas y responder de forma más contextualizada cuando el usuario haga referencia a información mencionada previamente.

VI. CONCLUSIONES

El uso de una arquitectura multiagente permite que se puedan abordar distintos tipos de consultas mediante sus respectivos agentes, evitando depender de una única fuente de información o de un único flujo de procesamiento. La incorporación de un orquestador facilita la selección dinámica de herramientas y agentes, permitiendo adaptar el comportamiento del sistema según la intención del usuario.

Asimismo, la integración de recuperación documental, búsqueda web, memoria conversacional y acceso controlado a servicios externos proporciona una base bastante flexible para construir respuestas más contextualizadas. Este enfoque favorece la incorporación de nuevas capacidades sin modificar significativamente la estructura general de la solución.

Por otro lado, la delegación del acceso a la base de datos transaccional a través de un servidor MCP garantiza el cumplimiento del principio de mínimo privilegio. La aplicación de técnicas de enmascaramiento y anonimización en esta capa intermedia previene eficazmente la exposición de datos sensibles al modelo de lenguaje, consolidando un diseño completamente seguro y protegiendo la privacidad de la información.

Finalmente, la arquitectura propuesta demuestra la utilidad de combinar modelos de lenguaje con herramientas especializadas y mecanismos de trazabilidad, permitiendo construir sistemas más controlables, auditables y adecuados para escenarios en los cuales la procedencia de la información y la transparencia del proceso de obtención de respuesta son aspectos relevantes.

REFERENCIAS

- [1] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
- [2] Y. Shoham and K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, 2020.
- [4] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [5] Google AI for Developers, "Gemini api documentation," <https://ai.google.dev/api>, 2026, accedido: 2026-06-05.
- [6] Tavily, "Tavily search api documentation," <https://docs.tavily.com/documentation/api-reference/endpoint/search>, 2026, accedido: 2026-06-05.
- [7] Chroma, "Chroma documentation," <https://docs.trychroma.com/>, 2026, accedido: 2026-06-05.
- [8] FastAPI, "Fastapi documentation," <https://fastapi.tiangolo.com/>, 2026, accedido: 2026-06-05.
- [9] Langfuse, "Langfuse observability documentation," <https://langfuse.com/docs/observability/overview>, 2026, accedido: 2026-06-05.